

Submission Worksheet

Submission Data

Course: IT202-008-S2026

Assignment: IT202 Milestone 1 - 2026

Student: Emperor G. (eg278)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 4/7/2026 10:51:13 AM

Updated: 4/7/2026 10:51:13 AM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone1-2026/eg278>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-1-2026/view/eg278>

Instructions

- Overview Link: <https://youtu.be/IDtqdaLwcVg>

1. Refer to Milestone1 of this doc:

<https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWWwfo/view>

2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone1 branch
 1. `git checkout Milestone1` (ensure proper starting branch)
 2. `git pull origin Milestone1` (ensure history is up to date)
5. Fill out the below worksheet
 - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
 - Ensure proper styling is applied to each page
 - Ensure there are no visible technical errors; only user-friendly messages are allowed
6. Once finished, click "Submit and Export"
7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 1. `git add .`
 2. `git commit -m "adding PDF"`
 3. `git push origin Milestone1`
 4. On Github merge the pull request from Milestone1 to qa
 5. On Github create a pull request from qa to prod and immediately merge. (This will trigger the prod deploy to make the heroku prod links work)
8. Upload the same PDF to Canvas
9. Sync Local
10. `git checkout qa`
11. `git pull origin qa`

Section #1: (2 pts.) Feature: User Will Be Able To Register A New Account

To Register A New Account

Progress: 100%

≡ Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

≡ Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

🖼 Part 1:

Progress: 100%

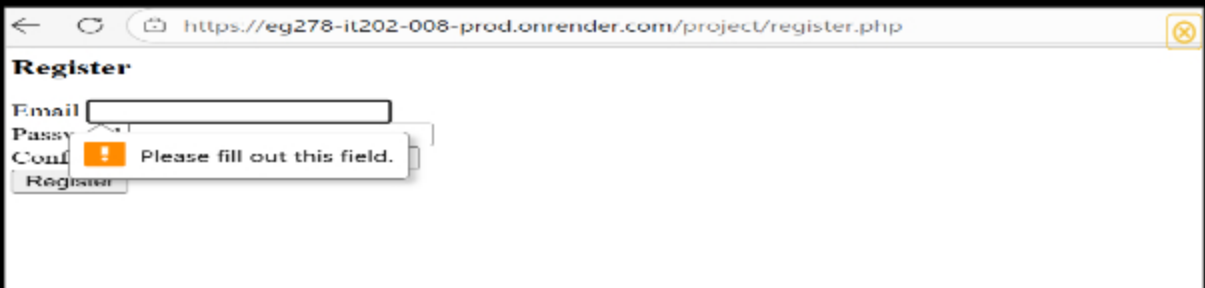
Details:

- Show the code related to the register page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)



A screenshot of a web browser showing a registration form titled "Register". The form has four input fields: "Email", "Password", "Confirm", and a "Register" button. The browser's address bar shows the URL "https://eg278-it202-008-prod.onrender.com/project/register.php".

the register page



A screenshot of the same registration form, but with a validation error. A red border is around the "Confirm" field, and a red error message "Please fill out this field." is displayed below it. The "Register" button is still visible.

required email

Register

Email

! Please include an '@' in the email address. 'hello' is missing an '@'.

invalid email format

Register

Email

Password

! Please lengthen this text to 8 characters or more (you are currently using 3 characters).

password too short

```
<h3>Register</h3>
<form onsubmit="return validate(this)" method="POST">
  <div>
    <label for="email">Email</label>
    <input id="email" type="email" name="email" required />
  </div>
  <div>
    <label for="username">Username</label>
    <input type="text" name="username" required maxlength="30" />
  </div>
  <div>
    <label for="pw">Password</label>
    <input type="password" id="pw" name="password" required minlength="8" />
  </div>
  <div>
    <label for="confirm">Confirm</label>
    <input type="password" name="confirm" required minlength="8" />
  </div>
  <input type="submit" value="Register" />
</form>
</script>
```

code snippet



Saved: 4/4/2026 2:15:30 AM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

First, the HTML validation occurs before JS or PHP. It avoids any forms of invalid from being inserted. Secondly, the Javascript Validation or also known as Client-side, it runs in the

inserted. Secondly, the JavaScript validation or also known as client side, it runs in the browser before the form is submitted. It checks for if the password contains at least one number and pass/confirm pass must match. Lastly, the PHP validation or the server-side. It checks for email is not empty, email is sanitized and valid, username follows required pattern, password is not empty, password is at least 8 characters, and password and confirm match. PHP shows flash error messages only if it fails, but if its valid the password is hashed and the user is in the database.

 Saved: 4/4/2026 2:15:30 AM

≡ Task #2 (0.33 pts.) - JS Validation (validate() function)

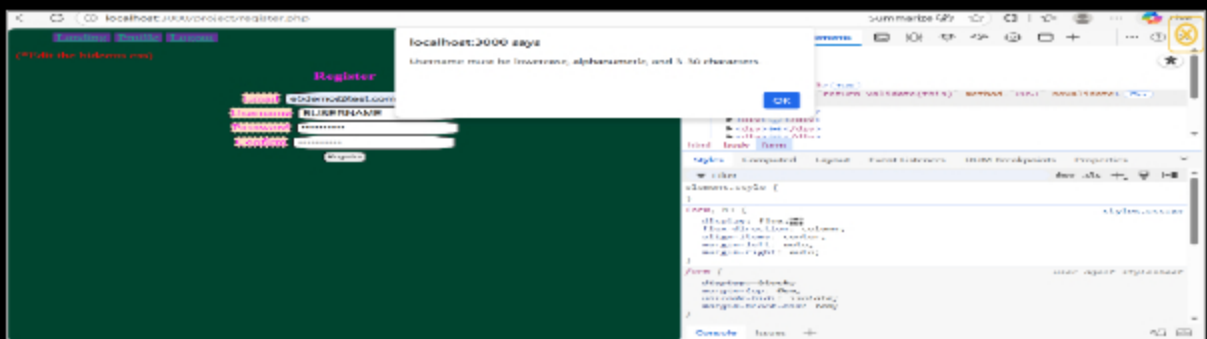
Progress: 100%

Part 1:

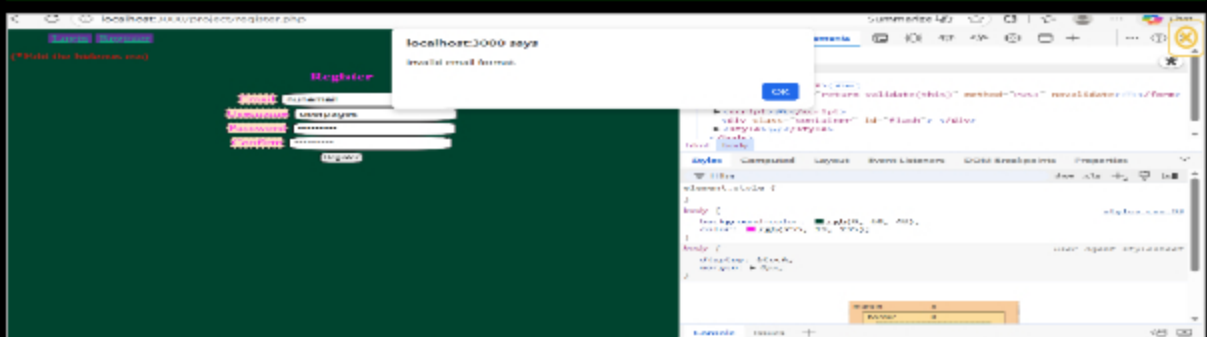
Progress: 100%

Details:

- Show the code related to the register page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm](you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

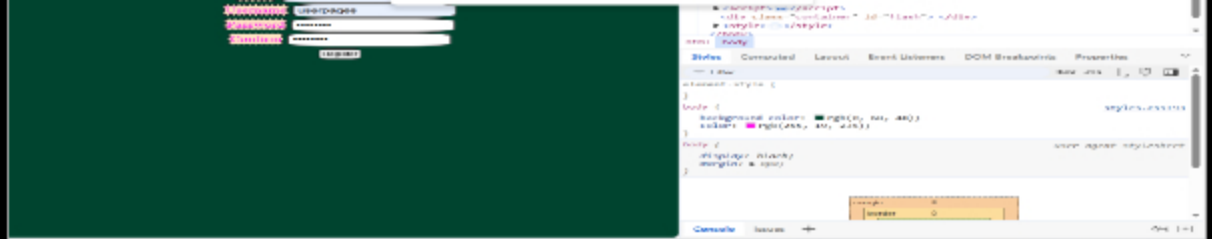


username format error

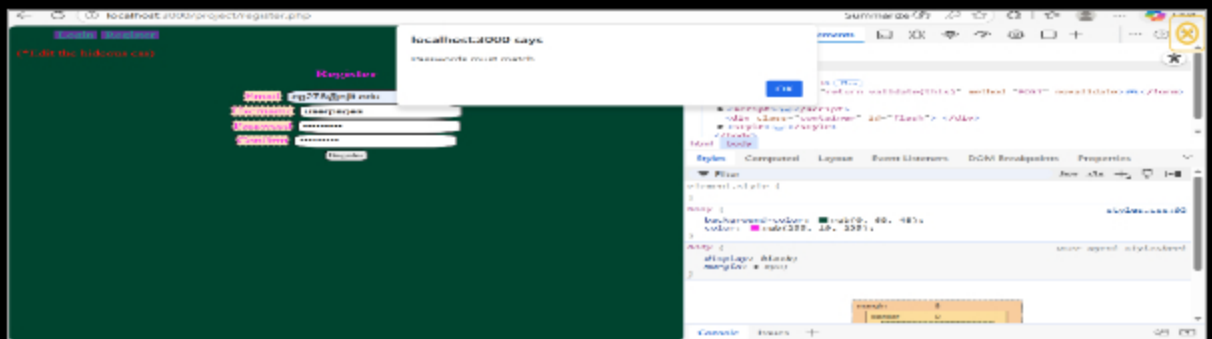


invalid email format





password missing number



password mismatch



Saved: 4/4/2026 10:22:43 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

First, when it comes to the username validation, it checks the username against Regex, it trims the extra spaces in the username, and makes sure the username is lowercase, alphanumeric, and contains 3-30 characters. Secondly, the email is also trimmed, it uses regex in order to check if the characters are before @, domain is valid, and that there is a dot with a domain extension. if however, the pattern is not aligned, it'll show "Invalid email format." Then there is the password match validation which compares the confirm field with the password, if it doesn't match, then it'll show "Passwords must match."



Saved: 4/4/2026 10:22:43 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the register page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the `validate()` function or use the provided helper script in the instructions
- Include the outputs related to username/email already in use

```
//TODO 3: add PHP Code
if (isset($_POST["email"], $_POST["password"], $_POST["confirm"], $_POST["username"])) {

    $email = $_POST["email"];
    $password = $_POST["password"];
    $confirm = $_POST["confirm"];
    $username = $_POST["username"];

    // TODO 4: validate/use
    $hasError = false;

    if (empty($email)) {
        //echo "Email must not be empty";
        flash("Email must not be empty.", "danger");
        $hasError = true;
    } else if (empty($password)) {
        // Sanitize and validate email
        $email = sanitize_email($email);
    } if (is_email($email)) {
        //echo "Invalid email address";
        flash("Invalid email address.", "danger");
        $hasError = true;
    } else if (empty($password)) {
        //echo "Password must be at least 8 characters long";
        flash("Password must be at least 8 characters long.", "danger");
        $hasError = true;
    } if (empty($confirm)) {
        //echo "Confirm password must match password";
        flash("Confirm password must match password.", "danger");
        $hasError = true;
    } if (empty($username)) {
        //echo "Username must be lowercase, alphanumeric, can only contain _ or -, and be between 3 to 30 characters";
        flash("Username must be lowercase, alphanumeric, can only contain _ or -, and be between 3 to 30 characters.", "danger");
        $hasError = true;
    }

    if ($hasError) {
        //echo "There are errors in your form";
        //echo "Please correct the errors and try again";
    } else {
        //echo "Registration successful";
        //echo "Please login";
    }
}
```

PHP Validation code

https://eg278-it202-008-prod.onrender.com/project/register.php

[Login](#)
[Register](#)

Invalid email address.

Register

Email

Username

Password

Confirm

PHP invalid email format

https://eg278-it202-008-prod.onrender.com/project/register.php

[Login](#)
[Register](#)

Username must be lowercase, alphanumeric, can only contain _ or -, and be between 3 to 30 characters

Register

Email

Username

Password

Confirm

PHP invalid username format

https://eg278-it202-008-prod.onrender.com/project/register.php

[Login](#)
[Register](#)

Password must be at least 8 characters long.

Register

Email

Username

Password

Confirm

password too short

Register

Passwords must match.

Email

Username

Password

Confirm

password mismatch



Saved: 4/5/2026 3:47:03 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

In PHP validation, the script will check if all the fields (email, username, password, confirm) were sent, otherwise it won't run successfully. In order to remove invalid/not safe characters, the email has to be sanitized, a flash message will appear if the email is empty, and if the email is not in a valid format then a flash message will say "Invalid email address". In regards to the username format, the script checks if the username is lowercase, alphanumeric, and can only contain "_" or "-", otherwise a flash message will display a username format error. Furthermore, the script checks if the password is not empty, if it is empty, there will be an error; it'll say "Password must be at least 8 characters long." Lastly, the script checks if the confirm is not empty, if so, it'll flash an error.



Saved: 4/5/2026 3:47:03 PM

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php`, zero credit if it does not)

- Include the render.com prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

[https://github.com/eg278-cmd/eg278-](https://github.com/eg278-cmd/eg278-it202-008-2026/pull/38)



URL

<https://github.com/eg278-crr>



[it202-008-2026/pull/38](https://github.com/eg278-cmd/eg278-it202-008-2026/pull/38)

URL #2

<https://eg278-it202-008-prod.onrender.com/project/register.php>



URL

[https://eg278-it202-008-prod.](https://eg278-it202-008-prod)



URL #3

<https://github.com/eg278-cmd/eg278-it202-008-2026/pull/38>



URL

<https://github.com/eg278-crr>



Saved: 4/5/2026 3:56:20 PM

Task #3 (0.67 pts.) - Database - Users table

Progress: 100%

Details:

- Add a screenshot of the database view from vs code showing a valid registered user (preferably a recent one during evidence capturing)

id	email	password	created	modified	username
1	eg278@njit.edu	\$2y\$12\$0c9G85oGzicboka	2026-04-01 14:00:49	2026-04-01 14:00:49	eg278-d3377e51d6793108s
4	emp25@njit.edu	\$2y\$12\$F5mq6PY4UrquvDx	2026-04-01 23:39:39	2026-04-06 20:36:55	testuser
5	e5demo@test.com	\$2y\$12\$0yKu9eRv8BywaT.M	2026-04-02 03:29:57	2026-04-02 03:29:57	e5demo
8	newemail23@test.com	\$2y\$12\$F0i5R6JdMn0CDUJx	2026-04-06 20:51:07	2026-04-06 21:19:56	admin

Users Table



Saved: 4/7/2026 10:51:13 AM

Section #2: (2 pts.) Feature: User Will Be Able To Login To Their Account

Progress: 100%

Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

☰ Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

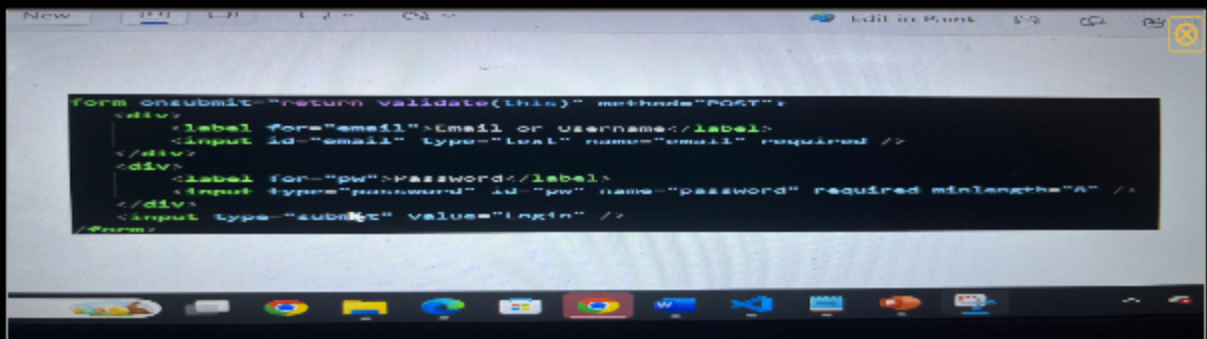
- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

📁 Part 1:

Progress: 100%

Details:

- Show the code related to the login page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

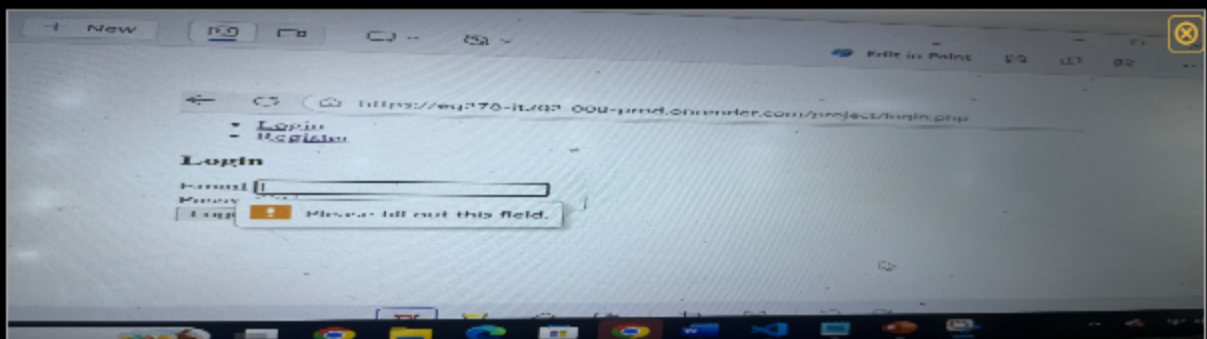


```

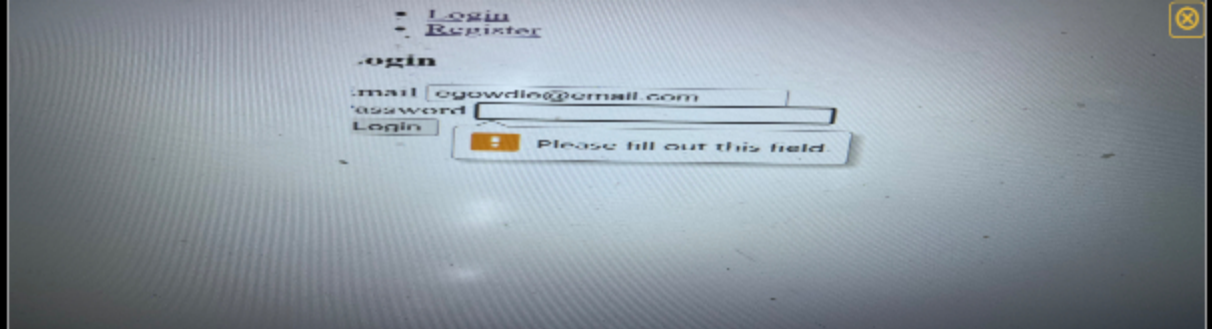
form onsubmit="return validate(this)" method="POST">
  <div>
    <label for="email">Email or Username</label>
    <input id="email" type="text" name="email" required />
  </div>
  <div>
    <label for="pw">Password</label>
    <input type="password" id="pw" name="password" required minlength="6" />
  </div>
  <input type="submit" value="Login" />
</form>

```

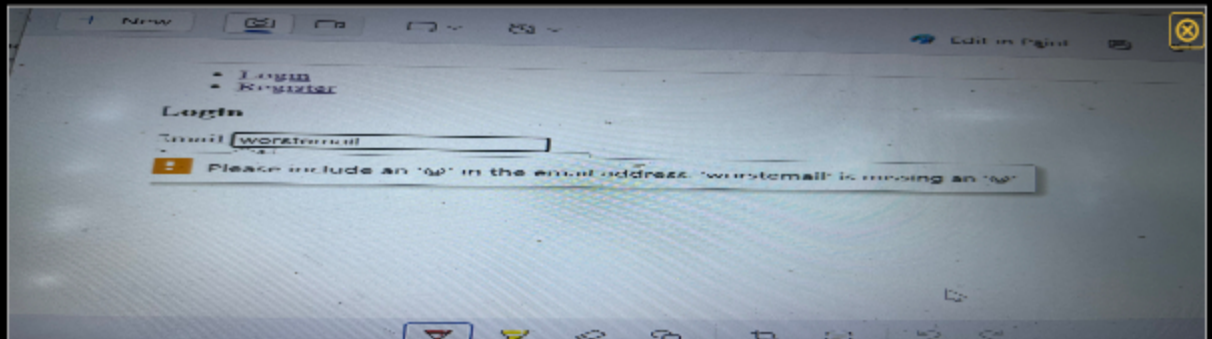
HTML Validation(login form)



HTML(empty/username)



HTML(empty password)



HTML(invalid email format)



Saved: 4/5/2026 8:38:02 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

In regards to the HTML validation, when the user clicks the login button, the browser will check both fields(email and password). If any these required fields are empty or invalid, for password, it'll say "Please fill out this field", or for email, it'll show "Please include an @ in the email address".



Saved: 4/5/2026 8:38:02 PM

≡ Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

🖼 Part 1:

Progress: 100%

Details:

- Show the code related to the login page's validate() function
- Show examples of each validation message [username format, email format, password format] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

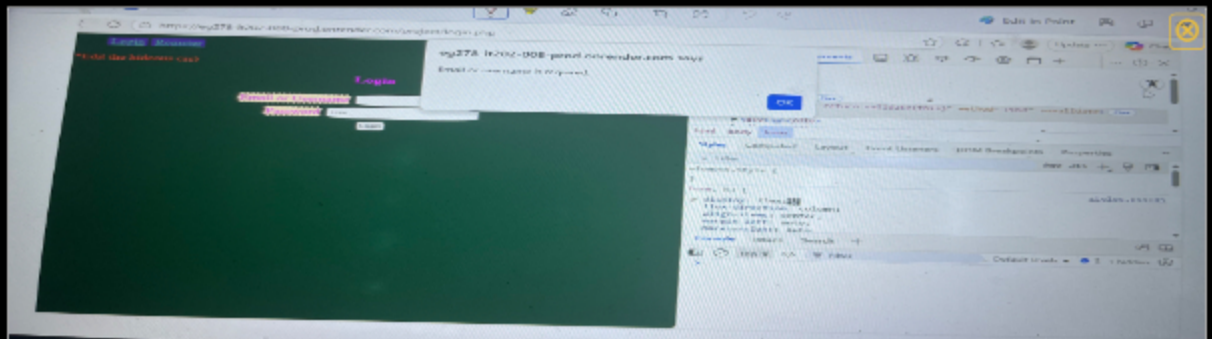
```
function validate(form) {
  //TODO 1: implement JavaScript validation (you'll do this on your own towards the end of Mile
  //ensure it returns false for an error and true for success
  let email = form.email.value.trim();
  let password = form.password.value.trim();

  // Email or username can't be empty
  if (email.length === 0) {
    alert("Email or username is required.");
    return false;
  }

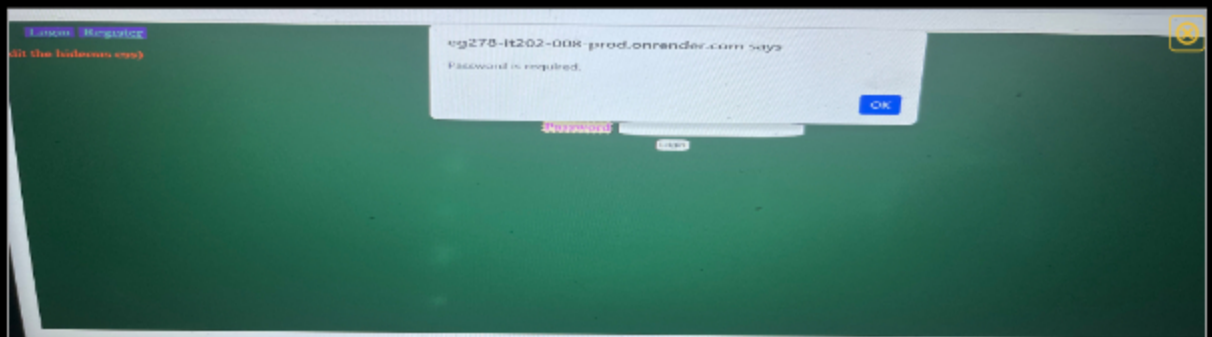
  if (password.length === 0) {
    alert("Password is required.");
    return false;
  }

  return true;
}
< #17-35 function validate(form)
```

JS Validation code(login)



JS(Empty username/email)



JS(Empty password)



Saved: 4/5/2026 8:34:39 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The JS validation runs once the user submits the form. It uses the `validate()` function to see if the user's input before the submitted form is valid. Once the user click Login, the `validate()` function will gather the information from both fields (email/username and password) and checks if its empty or invalid.



Saved: 4/5/2026 8:34:39 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the login page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the `validate()` function or use the provided helper script in the instructions
- Include the outputs related to user doesn't exist and php-side password mismatch (the one that checks against the DB hash)

```
// Basic form validation
if (isset($_POST["email"], $_POST["password"])) {
    // still leveraging the property as "email", but it can be a username
    $email = $_POST["email"];
    $password = $_POST["password"];
    // validate/use
    $hasError = false;

    if (empty($email)) {
        flash("Email is required.", "danger");
        $hasError = true;
    }

    if (filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
        // If it contains an @, treat it as an email
        // sanitize and validate email
        $email = sanitize_email($email);
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            flash("Invalid email address.", "danger");
            $hasError = true;
        }
    } else {
        // otherwise, treat it as a username
        $email = strtolower(trim($email));
        if (!is_valid_username($email)) {
            flash("Username must be lowercase, alphanumerical, and can only contain _ or -.", "danger");
            $hasError = true;
        }
    }
}
```

PHP Validation code

https://eq278-a202-036-prod.onrender.com/project/login.php

Learn React

(*Edit the tedious cas)

Email is required

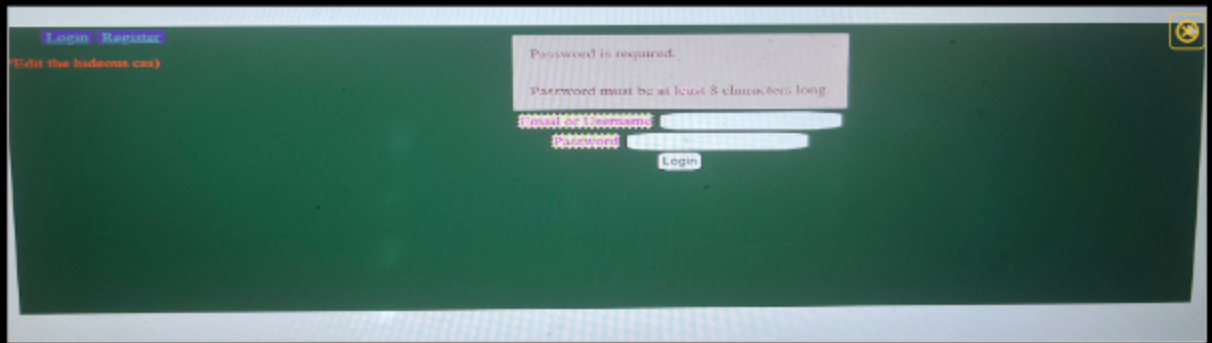
Username must be lowercase, alphanumerical, and can only contain _ or -

Email/Username

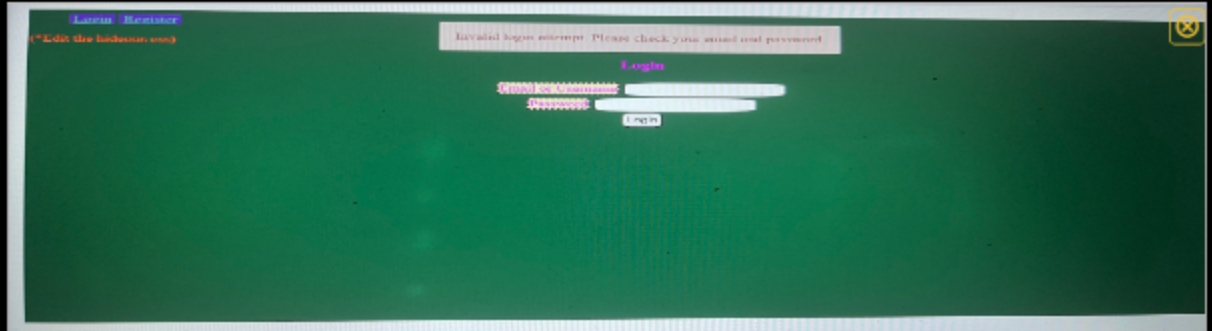
Password

Login

PHP error(empty email/username)



Empty password



Password mismatch



Saved: 4/5/2026 9:15:24 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The PHP validation is the process in the login page. It will only run once the form passes the HTML and JS validation. Once the both fields(email/username and password) is finished, the script executes the server-side and checks to see if both fields are not empty. After the login process, the PHP will show a flash message showing the login attempt was invalid depending on either field.



Saved: 4/5/2026 9:15:24 PM

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , , zero credit if it does not)
- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

[https://github.com/eg278-cmd/eg278-](https://github.com/eg278-cmd/eg278-it202-008-2026/pull/45)



URL

<https://github.com/eg278-crr>



[it202-008-2026/pull/45](https://github.com/eg278-cmd/eg278-it202-008-2026/pull/45)

URL #2

<https://eg278-it202-008-prod.onrender.com/project/login.php>



URL

[https://eg278-it202-008-prod.](https://eg278-it202-008-prod.onrender.com/project/login.php)



URL #3

<https://github.com/eg278-cmd/eg278-it202-008-2026/pull/45>



URL

<https://github.com/eg278-crr>



Saved: 4/5/2026 9:25:07 PM

Task #3 (0.67 pts.) - User Session Evidence

Progress: 100%

Details:

- From the render.com logs (Logs tab), capture the session error_log
- It should show the id, username, email, and roles

```
10:05:40 AM [9d7xn] [Tue Apr 07 14:05:40.296374 2026] [php:notice] [pid 17:tid 17] [client ::1:60548] Session: array
(\n 'user' => \n array (\n 'id' => 4,\n 'email' => 'emp25anjit.edu',\n 'username' =>
'testuser',\n 'roles' => \n array (\n 0 => \n array (\n 'name' =>
'Another',\n ),\n ),\n ),\n )
10:05:40 AM [9d7xn] ::1 - - [07/Apr/2026:14:05:40 +0000] "GET /project/landing.php HTTP/1.1" 200 752 "-"
"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/146.0.0.0 Safari/537.36 Edg/146.0.0.0"
```

Logs tab(id, username, email, roles)



Saved: 4/7/2026 10:17:17 AM

Section #3: (1 pt.) Feature: User Will Be Able

To Logout

Progress: 100%

≡ Task #1 (1 pt.) - Evidence

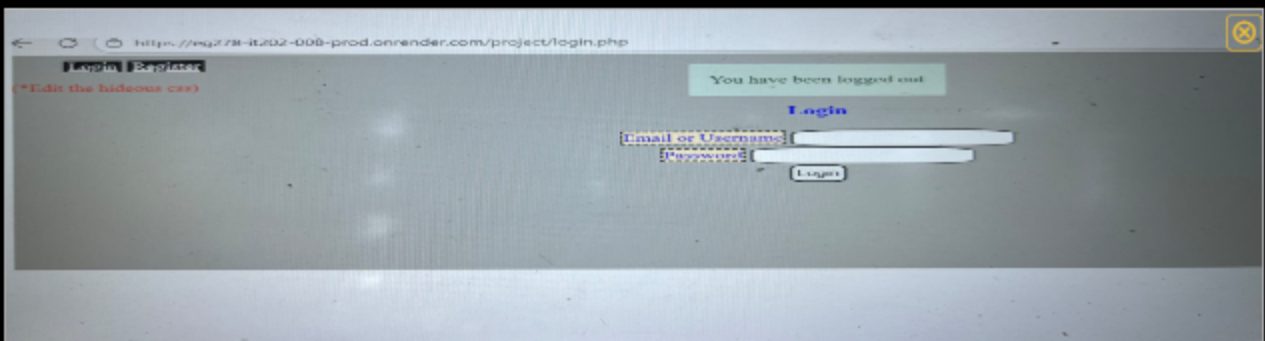
Progress: 100%

📁 Part 1:

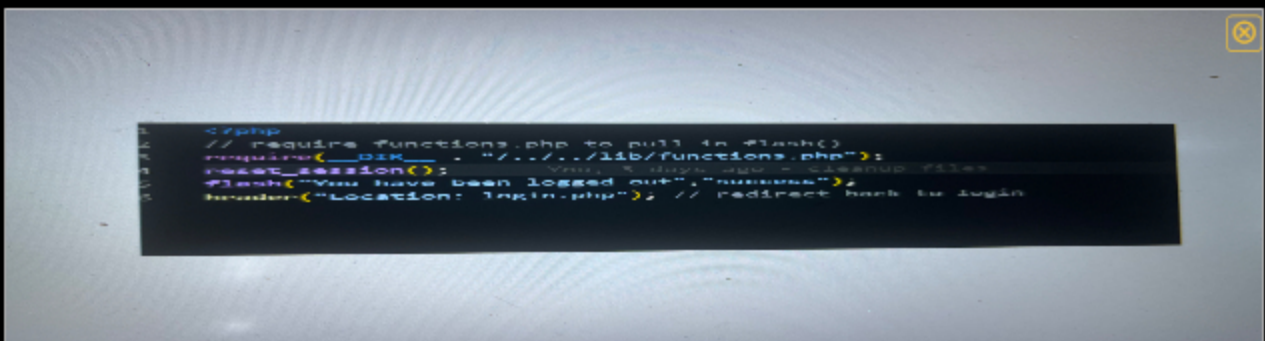
Progress: 100%

Details:

- Show the successful logout message (visit the proper file on Render.com qa after a manual deploy)
- Show the code snippet of the logout page



logout message



logout page code snippet



Saved: 4/5/2026 10:46:06 PM

🔗 Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1

<https://github.com/eq278-cmd/eq278->



URL

<https://github.com/eq278-cmd/eq>

Saved: 4/5/2026 10:46:06 PM

⇒ Part 3:

Progress: 100%

Details:

- Briefly explain how the logout process works and how the user is officially logged off

Your Response:

During the logout process, it clears the user's session data and redirects the user back to the login page. Once successful, the script calls `reset_session` to get rid of the already stored information such as the user's id, username, email, and roles. This will make the user unauthenticated. The session clears and then a flash message will say the user is fully logged out and goes back to the login page.

Saved: 4/5/2026 10:46:06 PM

Section #4: (2 pts.) Feature: Security Rules

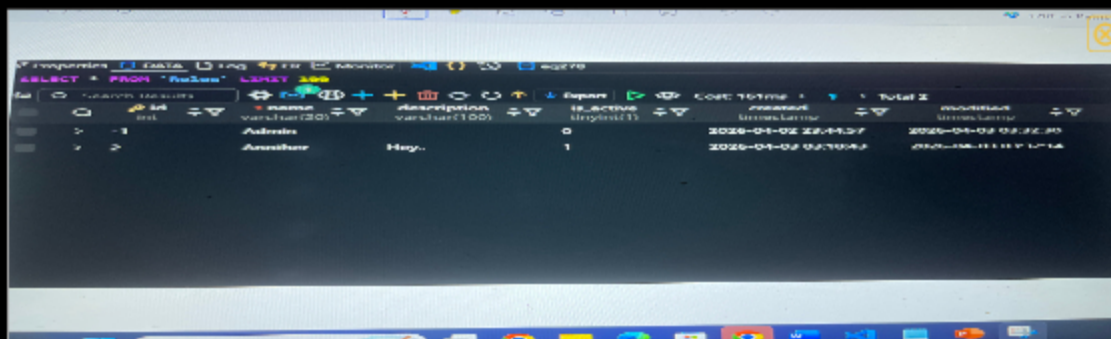
Progress: 100%

📁 Task #1 (1 pt.) - Database Tables

Progress: 100%

Details:

- From the vs code mysql tool, show both the ``Roles`` and ``UserRoles``



The screenshot shows the MySQL tool interface in VS Code. The SQL editor contains the query `SELECT * FROM `Roles``. The results pane displays the following data:

id	name	description	is_active	created	modified
1	Admin	Full access	1	2026-01-02 22:11:37	2026-01-02 22:11:37
2	User	Basic access	1	2026-01-02 22:11:37	2026-01-02 22:11:37

Roles data table



The screenshot shows the MySQL tool interface in VS Code. The SQL editor contains the query `SELECT * FROM `UserRoles``. The results pane displays the following data:

id	role_id	is_active	created	modified
1	1	1	2026-01-02 22:11:37	2026-01-02 22:11:37
2	2	1	2026-01-02 22:11:37	2026-01-02 22:11:37

2026-04-05 01:55:51	2026-04-05 01:55:51
2026-04-05 01:57:09	2026-04-05 01:57:09
2026-04-05 01:57:09	2026-04-05 01:57:09

UserRoles data table

 Saved: 4/5/2026 10:51:41 PM

Task #2 (1 pt.) - Demo Security Rules

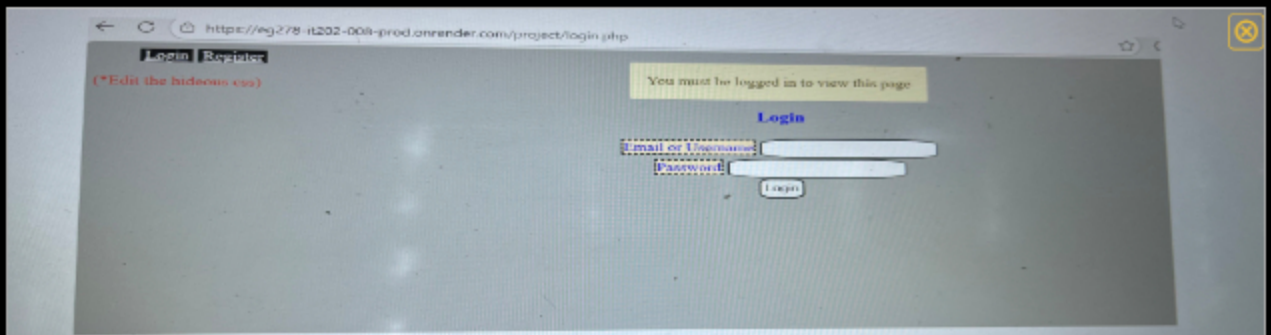
Progress: 100%

Part 1:

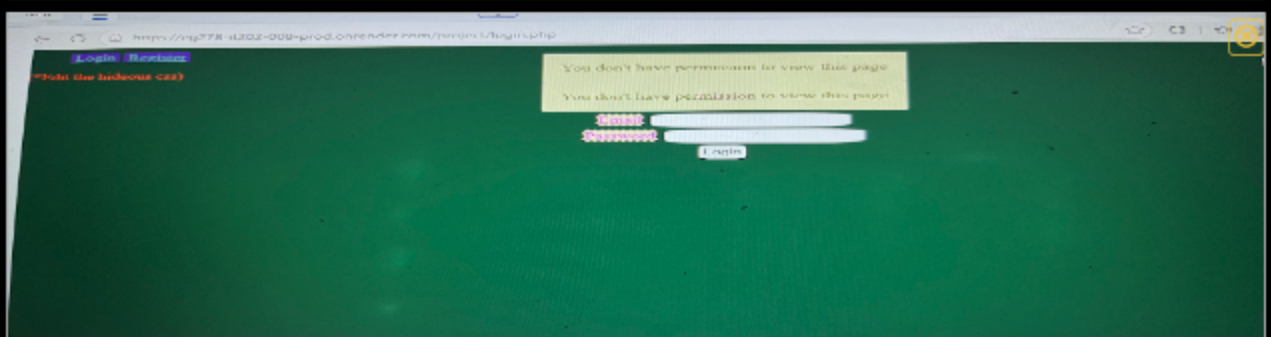
Progress: 100%

Details:

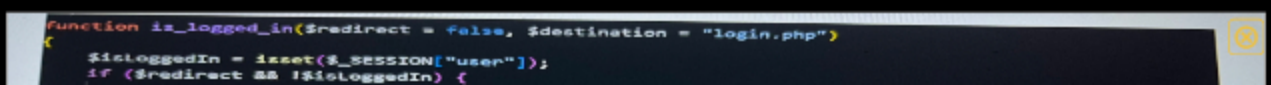
- Show the message related to needing to be logged in (i.e., try to manually
- Show the message related to not having the proper permission/role (i.e.,
- Include a snippet of the login check function
- Include a snippet of the role check function



access denied while logged out



no permission page



```

// if this triggers, the calling script won't receive a reply since die()/exit() terminates it
flash("You must be logged in to view this page", "warning");
$path = $destination;
// handle relative paths
if (!str_starts_with($path, "/")) {
    global $BASE_PATH; // pull from global scope of functions.php
    // ensure BASE_PATH ends with a slash so the url doesn't get malformed
    if (!str_ends_with($BASE_PATH, "/")) {
        $BASE_PATH .= "/";
    }
    $path = $BASE_PATH . $path; // prepend the base path
} // the else part is for absolute paths <= #17-24 if (str_starts_with($path, "/"))

die(header("Location: $path"));
} <= #12-27 if ($redirect && !$isLoggedIn)
return $isLoggedIn;

```

login check function snippet

```

function has_role($role)
{
    if (is_logged_in() && isset($_SESSION["user"]["roles"])) {
        foreach ($_SESSION["user"]["roles"] as $r) {
            if ($r["name"] === $role) {
                return true;
            }
        } <= #33-37 foreach ($_SESSION["user"]["roles"] as $r)
    } <= #32-38 if (is_logged_in() && isset($_SESSION["user"]["roles"]))
    return false;
}

```

role check function section

 Saved: 4/5/2026 11:45:58 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1

<https://github.com/eg278-cmd/eg278->



URL

<https://github.com/eg278-cmd/eg>

<https://github.com/eg278-cmd/eg278->
[it202-008-2326changes/8a614400bfd5c5a44d0eee5392a9212891a120eb#diff-eb6427eb71617f1cd6e2e105c556fc21f130e8a145c59fd174817d09cf816ebe](https://github.com/eg278-cmd/eg278-)

 Saved: 4/5/2026 11:45:58 PM

Part 3:

Progress: 100%

Details:

- Briefly explain how the login check code works
- Briefly explain how the role check code works

Your Response:

The way the login checks is that it confirms whether or not the user is authenticated through

checking to see if the `$_SESSION["user"]` exists. The page will have a flash message and redirects the user back to the login page if `$redirect = true` is called. As a result, it'll block any unfamiliar users who will attempt to access certain pages. The function will return true if the user is successfully logged in.



Saved: 4/5/2026 11:45:58 PM

Section #5: (2 pts.) Feature: User Profile

Progress: 100%

≡ Task #1 (1 pt.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

≡ Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

🖼 Part 1:

Progress: 100%

Details:

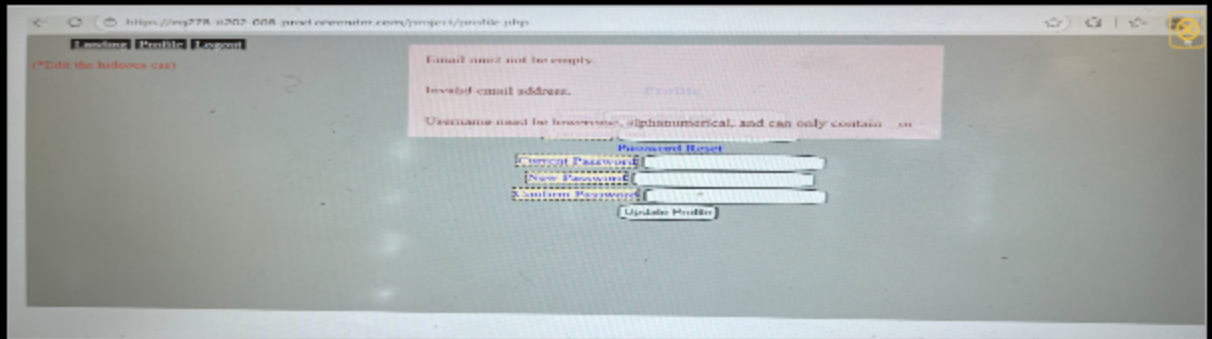
- Show the code related to the profile page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

```

<div profile />
<form method="POST" onsubmit="return validate(this);">
  <div class="mb-3">
    <label for="email">Email</label>
    <input type="email" name="email" id="email" value="{<php echo($email); ?>" />
  </div>
  <div class="mb-3">
    <label for="username">Username</label>
    <input type="text" name="username" id="username" value="{<php echo($username); ?>" />
  </div>
  <div class="mb-3">
    <div>DO NOT PRELOAD PASSWORD</div>
    <label for="password">Current Password</label>
    <input type="password" name="currentpassword" id="cp" />
  </div>
  <div class="mb-3">
    <label for="np">New Password</label>
    <input type="password" name="newpassword" id="np" />
  </div>
  <div class="mb-3">
    <label for="comp">Confirm Password</label>
    <input type="password" name="confirmpassword" id="comp" />
  </div>
  <input type="submit" value="Update Profile" name="save" />
</form>

```

HTML Validation(profile page)



HTML Validation(demo)



Saved: 4/6/2026 8:57:11 AM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The HTML validation of course runs first by the browser before the Javascript or PHP. The fields such as email and username use built-in HTML attributes like required, type="email", and minlength to make sure the user enters any information in the right format. Any required field that are remained empty, the browser will block anything from being submitted. I left every field empty and just clicked "Update Profile", this what lead for the present flash messages like "Email must not be empty, invalid email address, etc" to appear.



Saved: 4/6/2026 8:57:11 AM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

Part 1:

Progress: 100%

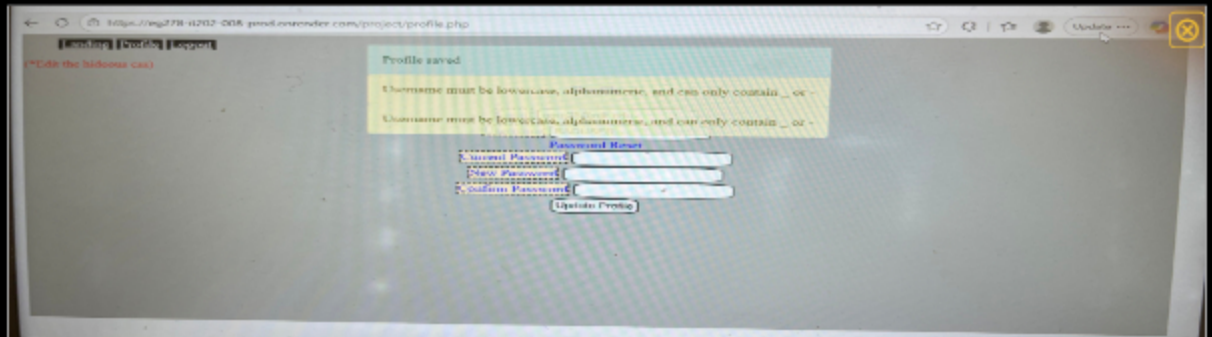
Details:

- Show the code related to the profile page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form

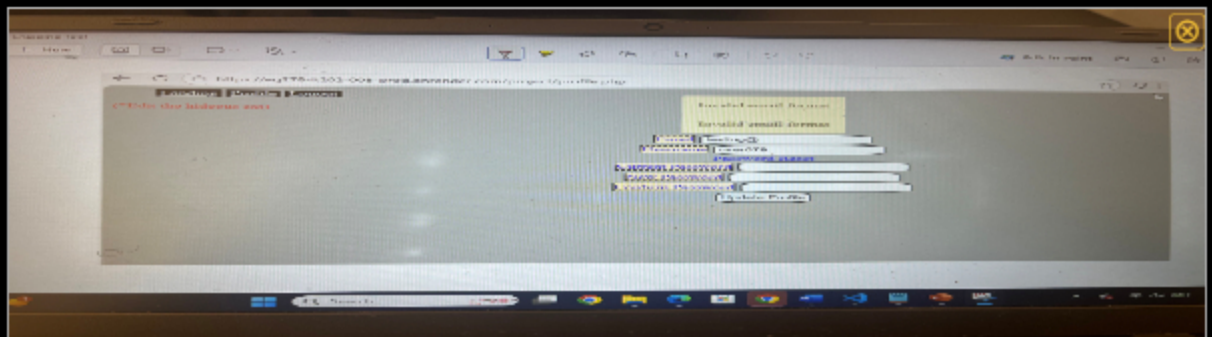
tag

```
function validate(form) {  
  let email = form.email.value.trim();  
  let username = form.username.value.trim();  
  let pw = form.newpassword.value;  
  let con = form.confirmPassword.value;  
  let isValid = true;  
  // TODO: add other client side validation....  
  // Example of using flash via javascript  
  // Used the flash container, create a new element, append it to  
  // NOTE: we'll extract the flash error to a function later  
  username = /^[a-z0-9-]{3,20}$/;  
  if (username.test(username)) {  
    flash(username must be lowercase, alphanumeric, and can only contain _ or - "warning");  
    isValid = false;  
  }  
  // email format  
  email = /^[a-zA-Z0-9]{1,254}@[a-zA-Z0-9]{1,40}\.([a-zA-Z]{2,5})$/;  
  if (email.test(email)) {  
    flash(email format error - Unrecognized email format, "warning");  
    isValid = false;  
  }  
  // Passwords must have at least 8 characters  
  pw = pw.length < 8 ? true : false;  
  con = con.length < 8 ? true : false;  
  if (pw || con) {  
    flash(password must be at least 8 characters, "warning");  
    isValid = false;  
  }  
  if (pw !== con) {  
    flash(passwords must match, "warning");  
    isValid = false;  
  }  
  return isValid;  
}
```

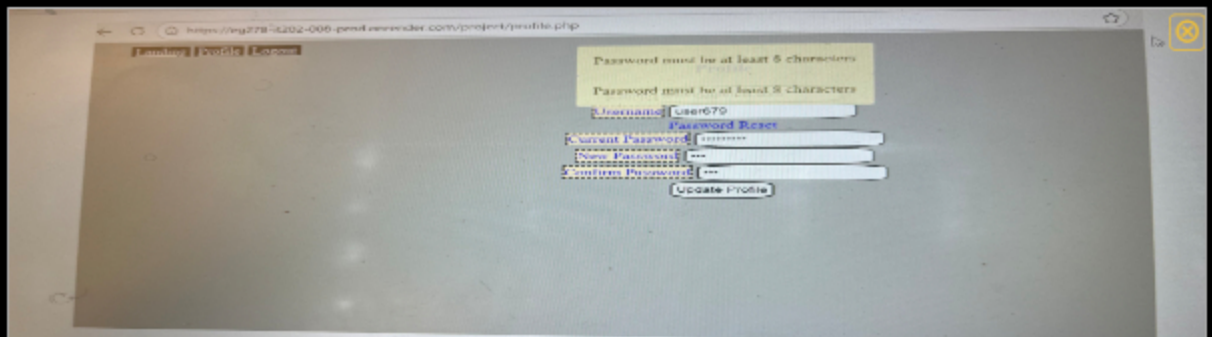
JS Validation code (profile page) part one



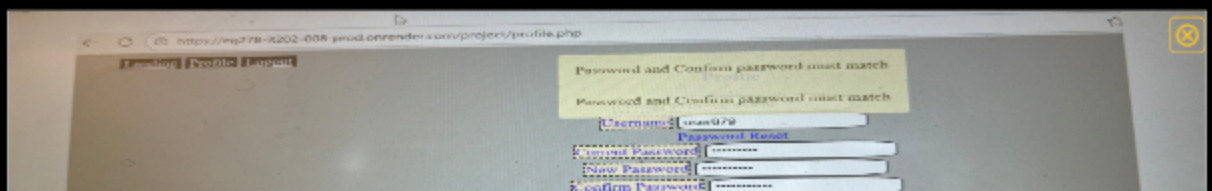
JS Validation(username format error)



JS Validation(email format error)



JS Validation(password too short)



JS Validation(password mismatch)

```
// Password form must have at least 8 characters
if (pw.length < 8 || pw.length > 16) {
    flash("Password must be at least 8 characters", "warning");
    isValid = false;
}

if (pw1 !== pw2) { // Check if password matches
    flash("Password and confirm password must match", "warning");
    isValid = false;
}

// Returning false will prevent the form from submitting
return isValid;
// Add this to the form's validate function
require_ajax = true; // ../passwords/flash.php
}
```

JS Validation code part two



Saved: 4/6/2026 6:54:58 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

In the Javascript validation, once the user clicks "Update Profile", the validate function() examines each required field and blocks the form from submitting if anything is invalid. The script will only run if everything matches the information for each field; username can only be lowercase and contains letters, numbers, "_", or "-", passwords should contain at least 8 characters and both "New Password" and "Confirm Password" must match.



Saved: 4/6/2026 6:54:58 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's PHP validation
- Show examples of each validation message [username format, email format,

password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the `validate()` function or use the provided helper script in the instructions

```
// handle email/username update
if (isset($_POST["email"], $_POST["username"])) {
    $new_email = $_POST["email"];
    $new_username = $_POST["username"];
    $hasError = false;
    // validate format
    if (empty($new_email)) {
        //echo "Email must not be empty";
        $hasError = true;
    }
    // Sanitize and validate email
    $new_email = sanitize_email($new_email);
    if (!is_valid_email($new_email)) {
        //echo "Invalid email address";
        $hasError = true;
    }
    if (!is_valid_username($new_username)) {
        //echo "Username must be lowercase, alphanumerical, and can only contain _ or -";
        $hasError = true;
    }
}
```

PHP Validation code part 1

```
//check/update password
if (isset($_POST["currentPassword"], $_POST["newPassword"], $_POST["confirmPassword"])) {
    $current_password = $_POST["currentPassword"];
    $new_password = $_POST["newPassword"];
    $confirm_password = $_POST["confirmPassword"];
    // require all 3 to be set before attempting to process
    $can_update = !empty($current_password) && !empty($new_password) && !empty($confirm_password);
    if ($can_update) {
        // check that new matches confirm (i.e.. no typos)
        if (!is_valid_confirm($new_password, $confirm_password)) {
            //echo "New passwords don't match";
            $hasError = true;
        }
        //validate current password against password rules
        if (!is_valid_password($current_password)) {
            //echo "Password too short";
            $hasError = true;
        }
        if (!is_valid_password($new_password)) {
            //echo "Password must be at least 6 characters long";
            $hasError = true;
        }
        if ($hasError) {
            // fetch current hash
            try {
                $db = getDB();
                $stmt = $db->prepare("SELECT password FROM Users where id = ?");
                // using get_user_id() in this block to ensure we don't mistakenly allow changing
                $stmt->execute(["id" => get_user_id()]);
                $result = $db->fetch(PDO::FETCH_ASSOC);
            } catch (PDOException $e) {
                //echo "Database error: " . $e->getMessage();
            }
        }
    }
}
```

PHP Validation(password)

https://eg278-4202-008-prod.onrender.com/project/profile.php

Landing Profile Logout

Username must be lowercase, alphanumerical, and can only contain _ or -

Email: emp25@qst.edu

Username: user679

Password Reset

Current Password

New Password

Confirm Password

Update Profile

PHP Validation(username format error)

https://eg278-4202-008-prod.onrender.com/project/profile.php

Landing Profile Logout

Invalid email address

Email: emp25@qst.edu

Username: user679

Password Reset

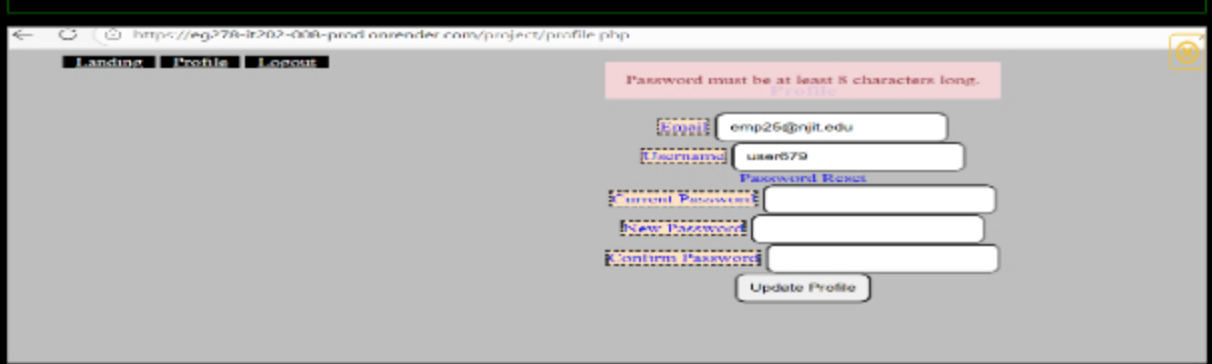
Current Password

New Password

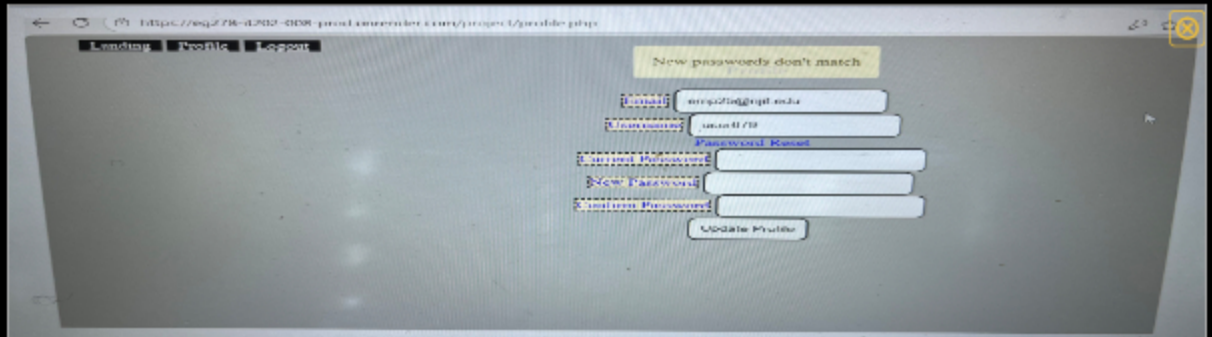
Confirm Password

Update Profile

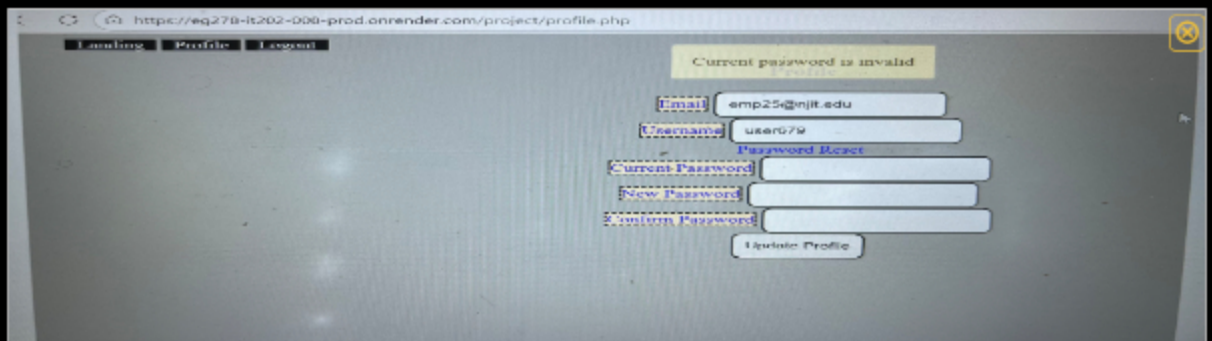
Email format error



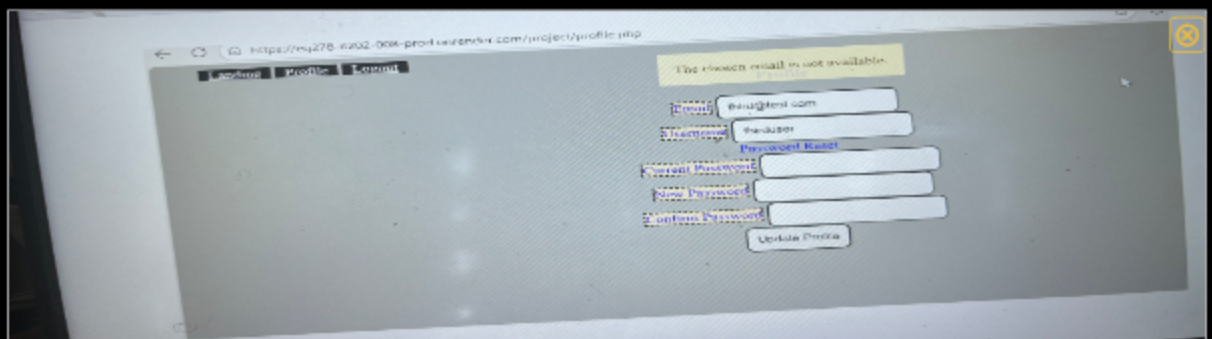
password error



password mismatch



Invalid current password



Existing email



Saved: 4/6/2026 5:27:33 PM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

During the PHP validation, when the user clicks "Update Profile", PHP will receive the information from all the submitted fields and checks if they're valid or invalid. It then verifies that the email is not empty, sanitizes the email, and makes sure it's in the correct format; this involves the username, the password, and the other password fields (current password and new password).



Saved: 4/6/2026 5:27:33 PM

Task #2 (1 pt.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , , zero credit if it does not)
- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

https://github.com/eg278-cmd/eg278-cmd/blob/it202-008/2026/milestone1/public_html/project/profile.php



URL

<https://github.com/eg278-crr>



https://github.com/eg278-cmd/eg278-cmd/blob/it202-008/2026/milestone1/public_html/project/profile.php

URL #2

<https://learn.ethereallab.app/assignment/v3/IT20830086/vv>



URL

<https://learn.ethereallab.app/>



URL #3

<https://github.com/eg278-cmd/eg278-cmd/blob/it202-008/2026/changes/6d3f088fca6f00ef60d902b99bdccec78ef239ee>



URL

<https://github.com/eg278-crr>



Saved: 4/6/2026 5:34:27 PM

Section #6: (1 pt.) Misc

Progress: 100%

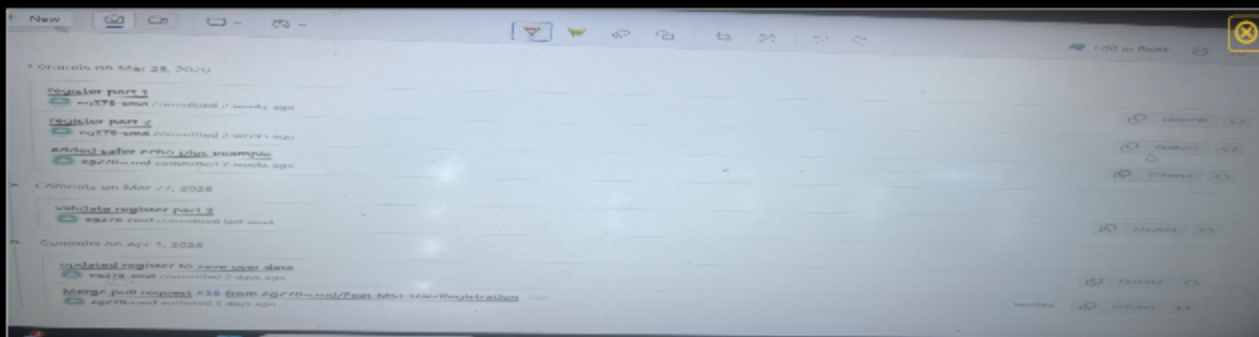
Task #1 (0.33 pts.) - Github Details

Progress: 100%

Part 1:

Details:

From the Commits tab of the Pull Request screenshot the commit history (it may not be possible to capture all, just grab a reasonable chunk)



Commits tab history part one



Commits tab history part 2



 Saved: 4/6/2026 6:21:58 PM

🔗 Part 2:

Details:

Include the link to the Pull Request (should end in `/pull/#`) (Milestone1 into qa)

URL #1

<https://github.com/eq278-cmd/eq278-it202-008-2021646/>



UH

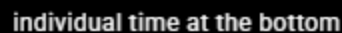
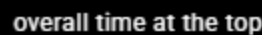
<https://github.com/eg278-cmd/eg>



 Saved: 4/6/2026 6:21:58 PM

Task #2 (0.33 pts.) - WakaTime - Activity

- Visit the [WakaTime.com](https://wakatime.com) Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



Based on the entire milestone1, I've learned how to demonstrate that different validation formats within PHP, JS, and HTML. I learned the basic concepts how a login sysstern works through the use redirects, updating profile, password hashing, and session handling. Also, lastly how to debug a developer with dev

hashing, and session handling. Also, lastly how to debug a developer with dev tools, flash messages, error logs, etc



Saved: 4/6/2026 6:37:01 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of the assignment was to edit the background and field size with colors in CSS.



Saved: 4/6/2026 6:38:26 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of the assignment was trying to update both the login.php and profile.php



Saved: 4/6/2026 6:39:22 PM